

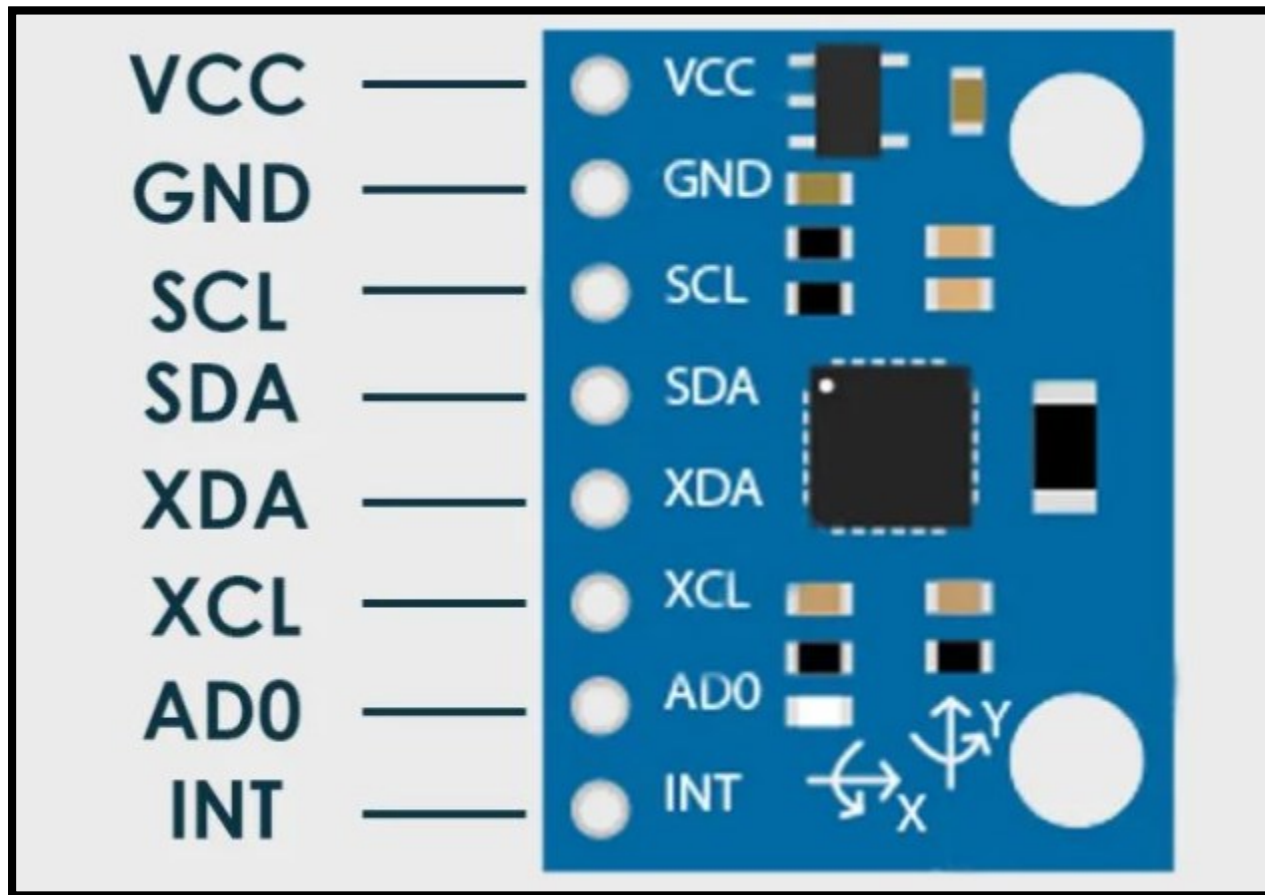
ARDUINO MOTION SENSOR OXYMETER

University of Alexandria, Faculty of Engineering,
Computer and Systems Engineering Department.
CSE343 Embedded Systems Spring 2026

MPU 6050

- IMU device that stands for Inertial Measurement Unit.
- It is a six-axis motion tracking device that calculates a three-axis accelerometer and three-axis gyroscope data.
- MPU 6050 also includes a temperature sensor.

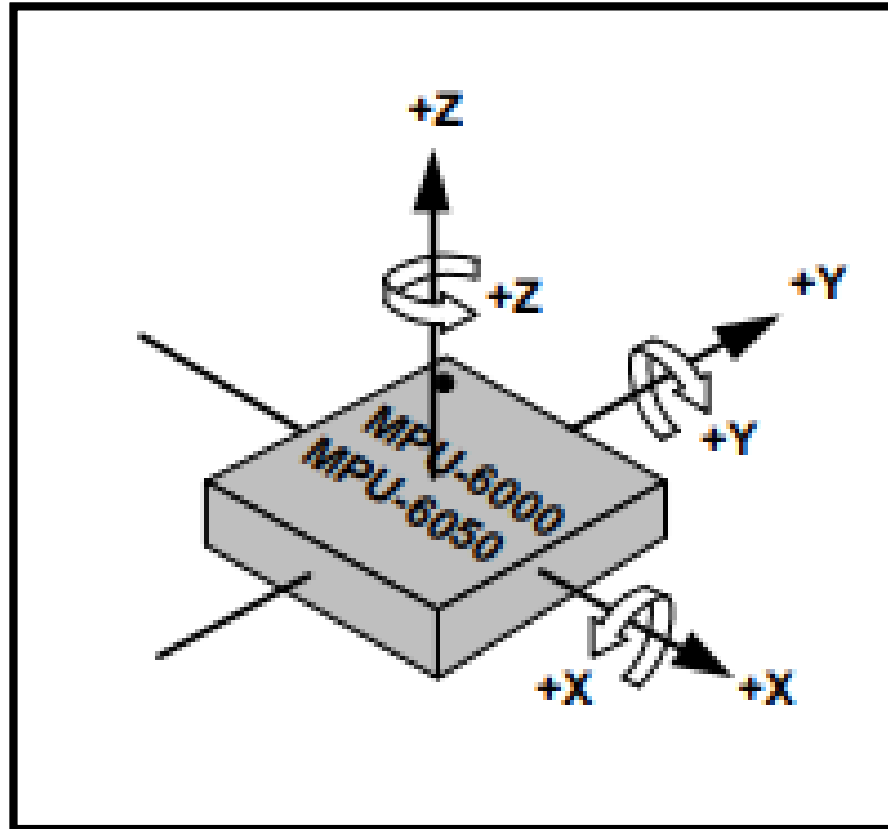
MPU 6050 Schematic



MPU 6050 Pins

- **VCC:** This pin is used to provide power(5v /3.3v)
- **GND:** This pin is used to provide 0V.
- **SDA:** This pin is used for obtaining data serially from the sensor.
- **SCL:** This pin is used for serial clock input.

MPU 6050 Orientation



MPU 6050 Accelerometer Specs

ACCELEROMETER SENSITIVITY				
Full-Scale Range	AFS_SEL=0		±2	g
	AFS_SEL=1		±4	g
	AFS_SEL=2		±8	g
	AFS_SEL=3		±16	g
ADC Word Length	Output in two's complement format		16	bits
Sensitivity Scale Factor	AFS_SEL=0		16,384	LSB/g
	AFS_SEL=1		8,192	LSB/g
	AFS_SEL=2		4,096	LSB/g
	AFS_SEL=3		2,048	LSB/g
Initial Calibration Tolerance			±3	%
Sensitivity Change vs. Temperature	AFS_SEL=0, -40°C to +85°C		±0.02	%/°C
Nonlinearity	Best Fit Straight Line		0.5	%
Cross-Axis Sensitivity			±2	%

MPU 6050 Subset Registers

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

MPU 6050 Starter Code

```
const int MPU_ADDR = 0x68; // I2C address of the MPU-6050. If AD0 pin is set to HIGH, the I2C address will be 0x69.

int16_t accelerometer_x, accelerometer_y, accelerometer_z; // variables for accelerometer raw data
int16_t gyro_x, gyro_y, gyro_z; // variables for gyro raw data
int16_t temperature; // variables for temperature data

void setup() {
  Serial.begin(9600);
  Wire.begin();
  Wire.beginTransmission(MPU_ADDR); // Begins a transmission to the I2C slave (GY-521 board)
  Wire.write(0x6B); // PWR_MGMT_1 register
  Wire.write(0); // set to zero (wakes up the MPU-6050)
  Wire.endTransmission(true);
}

void loop() {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H) [MPU-6000 and MPU-6050 Register Map and Descriptions Revision 4.2, p.40]
  Wire.endTransmission(false); // the parameter indicates that the Arduino will send a restart. As a result, the connection is kept active.
  Wire.requestFrom(MPU_ADDR, 7*2, true); // request a total of 7*2=14 registers

  // "Wire.read()<<8 | Wire.read();" means two registers are read and stored in the same variable
  accelerometer_x = Wire.read()<<8 | Wire.read(); // reading registers: 0x3B (ACCEL_XOUT_H) and 0x3C (ACCEL_XOUT_L)
  accelerometer_y = Wire.read()<<8 | Wire.read(); // reading registers: 0x3D (ACCEL_YOUT_H) and 0x3E (ACCEL_YOUT_L)
  accelerometer_z = Wire.read()<<8 | Wire.read(); // reading registers: 0x3F (ACCEL_ZOUT_H) and 0x40 (ACCEL_ZOUT_L)
  temperature = Wire.read()<<8 | Wire.read(); // reading registers: 0x41 (TEMP_OUT_H) and 0x42 (TEMP_OUT_L)
  gyro_x = Wire.read()<<8 | Wire.read(); // reading registers: 0x43 (GYRO_XOUT_H) and 0x44 (GYRO_XOUT_L)
  gyro_y = Wire.read()<<8 | Wire.read(); // reading registers: 0x45 (GYRO_YOUT_H) and 0x46 (GYRO_YOUT_L)
  gyro_z = Wire.read()<<8 | Wire.read(); // reading registers: 0x47 (GYRO_ZOUT_H) and 0x48 (GYRO_ZOUT_L)

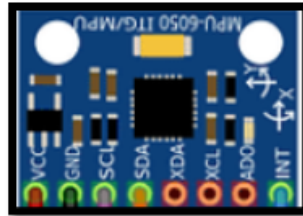
  //TODO: Your code goes here

  // delay
  delay(2);
}
```


Variant Subproblem

Part One: Basic Recording.

You are required to implement a software-hardware strategy to measure the acceleration angle. Initially the accelerometer is placed horizontally at rest as shown below(plan view).



As the time proceeds, start moving the accelerometer with an angle that deviates horizontally and then vertically. Print the angle with x-axis, y-axis, and z-axis using serial print. You should adjust the plots for the angles to be aligned under eachother with 400 units space in between and keep the plots for the angle to be from 0 to 360 degrees. In addition, plot the magnitude of the resultant force after decreasing the bias g (16000 units magnitude of acceleration in z-axis) and normalize the graph to be from (0 to 360) with 400 units shift from the last angle printed. This is done in order to keep all the plots shown without overlapping and having equal areas of plotting over the plotter.

Variant Subproblem(2)

Part Two: Data Stream Recording for Later Use

The basic problem in part one is that we can only process the data instantly, we need to adapt the strategy for later use. You are required to implement one of the following variants:

Variant One: Excel Data Streamer

- Record the last 100 readings using microsoft office excel data streamer(use Microsoft office 365 provided by our university or if you have a version of microsoft office 2016+). Use excel plotting to visualize the streamed data.
- Download and install the data streamer. You can check the foolowing link for download:
- <https://www.microsoft.com/en-us/download/details.aspx?id=56976>
- Disconnect the serial printer/plotter and now you can use the excel data streamer to record the readings.

Variant Subproblem(3)

Variant Two: Record the last 10 readings on Arduino Builtin EEPROM

- Modify the provided code to save the last 10 readings on the EEPROM. Process the data to get the steady state acceleration (average of the 10 readings of the resultant acceleration). If the resultant acceleration exceeds **1.2g**, determine the force that caused this spike in acceleration.

Variant Three: Record the readings over the last 20 seconds on an external SD card using SD-Card

Module

- Modify the hardware interfacing to use an external SD card module.
- Modify the code to save the readings over the last 20 seconds on the SD card.

Excel Data Streamer

- You should download the data streamer for Microsoft office excel(2016+ or Microsoft 365).
 - <https://www.microsoft.com/en-us/download/details.aspx?id=56976>
- Installation details
 - <https://support.microsoft.com/en-us/office/what-is-data-streamer-1d52ffce-261c-4d7b-8017-89e8ee2b806f>
- Adapted Arduino Sketch Modified version of “Michael Schoeffler”-2017 implementation.
 - <https://drive.google.com/file/d/1llgqO5P18jwzxryKafivj5AAwvxVV0x1/view?usp=sharing>

Excel Data Streamer(2)

The screenshot displays the Excel Data Streamer ribbon with the following sections:

- 1 Data Sources:** Disconnect Device, Import Data File.
- 2 Data Streaming:** Start Data, Stop Data.
- 3 Data Recording:** Record Data, Stop Recording, Capture Visualization.
- Advanced:** Reset Data, Advanced.
- Help:** Help.

The 'Data In (From Source)' section shows a table with the following data:

	A	B	C	D	E	F	G	H	I	J	K
4	Time	CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8	CH9	CH10
5	12:01.3	1484	-418	13830							
6	Historical Data										
7	Time	CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8	CH9	CH10
8	12:01.0	1458	-462	13750							
9	12:01.0	1398	-400	13798							
10	12:01.0	1412	-464	13780							
11	12:01.0	1446	-404	13784							
12	12:01.1	1482	-430	13790							
13	12:01.1	1440	-446	13762							
14	12:01.1	1454	-452	13784							
15	12:01.2	1474	-474	13724							
16	12:01.2	1450	-490	13758							
17	12:01.2	1412	-440	13758							
18	12:01.2	1424	-482	13716							
19	12:01.3	1430	-442	13696							
20	12:01.3	1470	-428	13758							
21	12:01.3	1430	-502	13798							
22	12:01.3	1484	-418	13830							

Graphical Simulation Based on Motion Sensing using Processing

- You should install the Processing IDE.
 - <https://processing.org/>
- Install the “ToxicLibs” library from the IDE Contribution manager.
- Arduino sketch by “Jeff Rowberg”.
 - <https://drive.google.com/file/d/1QSgXaDVEGus9KQgfdRPyOpSD7wM-gzsr/view?usp=sharing>
- Processing sketch by “Jeff Rowberg”.
 - https://drive.google.com/file/d/1iMJTx9Uwe_QinW8SfzLvY6Q7wEealkuu/view?usp=sharing

Oxymeter

- A pulse oximeter is basically a device which can measure your pulse and oxygen saturation in your blood.
- Usually this sensor consists of two LEDs emitting light:
 - one in Red spectrum (650nm) and the other one in Infrared (950nm).
 - This sensor is placed on your finger or earlobe, essentially anywhere where the skin is not too thick so both light frequencies can easily penetrate the tissue. Once both of them are shined through your finger for example, the absorption is measure with a photodiode.
 - Depending on the amount of oxygen you have in your blood the ratio between the absorbed red light and IR led will be different.

Connections

MAX30102 Pin	Arduino Pin
VIN	5V
GND	GND
SCL	A5
SDA	A4

Oxymeter Starter Code

```
#include "MAX30100.h"

MAX30100* pulseOxymeter;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Pulse oxymeter test!");

  pulseOxymeter = new MAX30100();
  pinMode(2, OUTPUT);
}

void loop() {
  pulseoxymeter_t result = pulseOxymeter->update();

  if( result.pulseDetected == true )
  {
    Serial.println(result.SaO2);
  }
}
```

References

Beside Lecture Notes, we used the following additional resources:

- **Interfacing MPU6050 to Arduino**
 - <https://robu.in/mpu6050-with-arduino/>
- **MPU6050 Datasheet**
 - https://www.alldatasheet.com/view.jsp?Searchword=Mpu-6050%20datasheet&qclid=CjwKCAjwuYWSBhByEiwAKd_n_hDGJ34mpdblcmrrd6VNq1MlqA-le62ay3KLJjbu7_LVR4C2_c_uyRoCxdwQAvD_BwE
- **Oxymeter Mechanism of Work**
 - [oximeter intro.pdf - Google Drive](#)
- **Developing Arduino Library**
 - [Arduino - LibraryTutorial - Reader Mode.pdf - Google Drive](#)